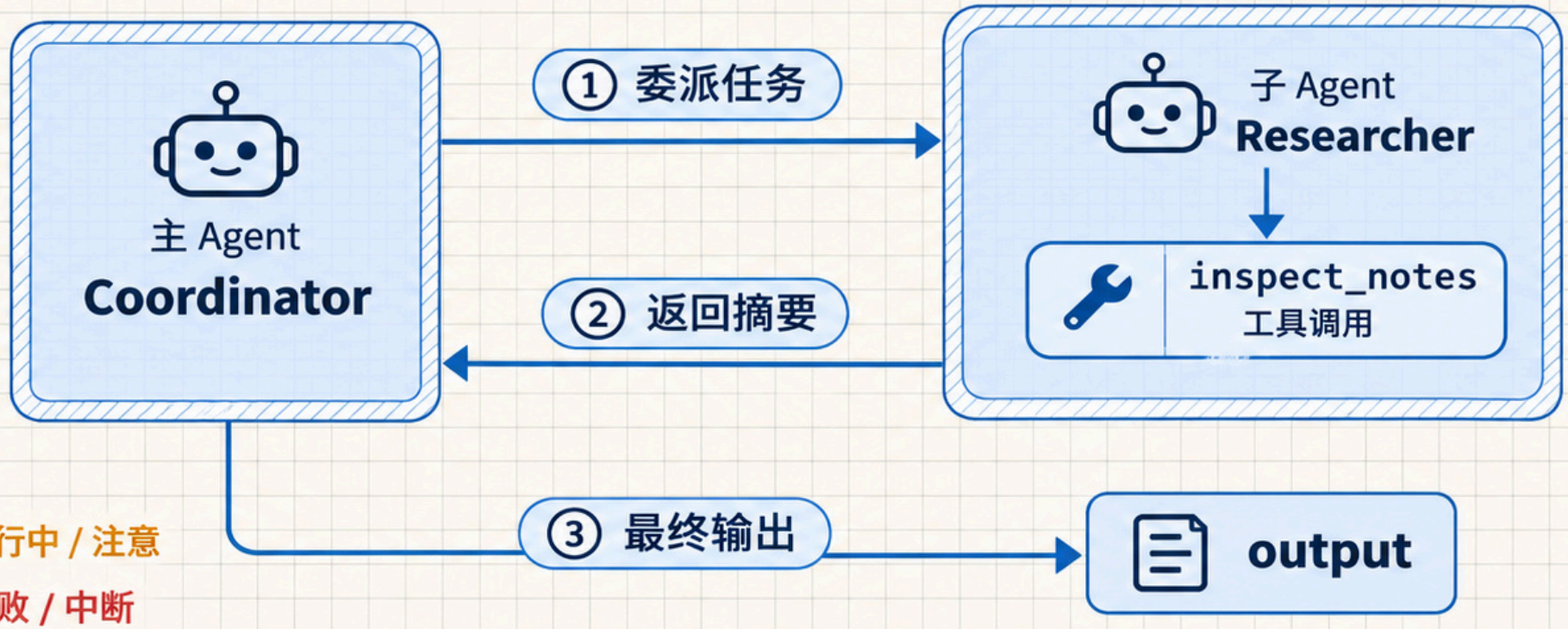
Deep Agents 实战

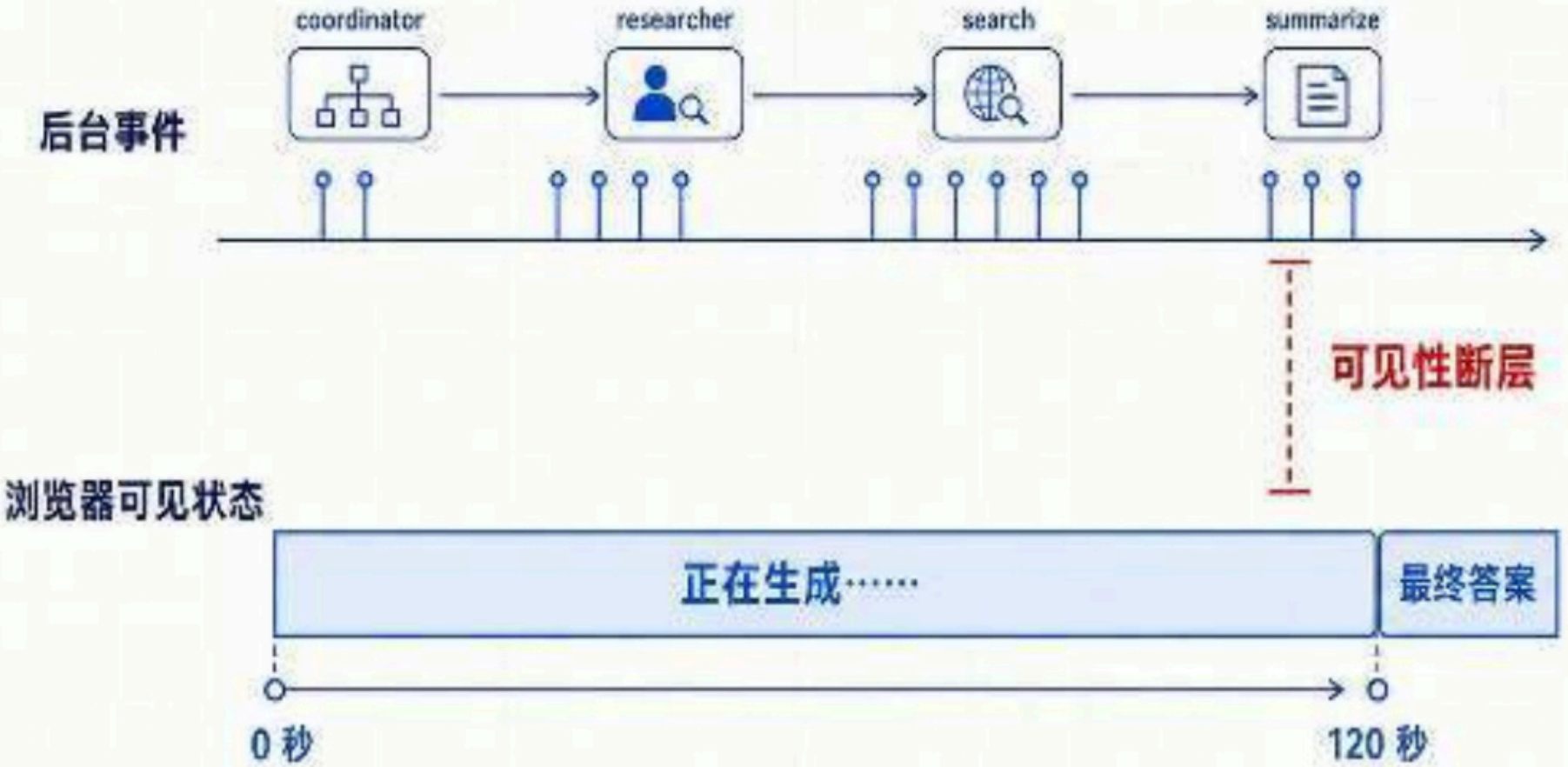
第 14 章：Streaming

实时观察主 Agent、子 Agent 与工具调用



最终答案能返回，页面依然像卡住了

后台已启动 researcher 并调用搜索；前台两分钟只有“正在生成……”



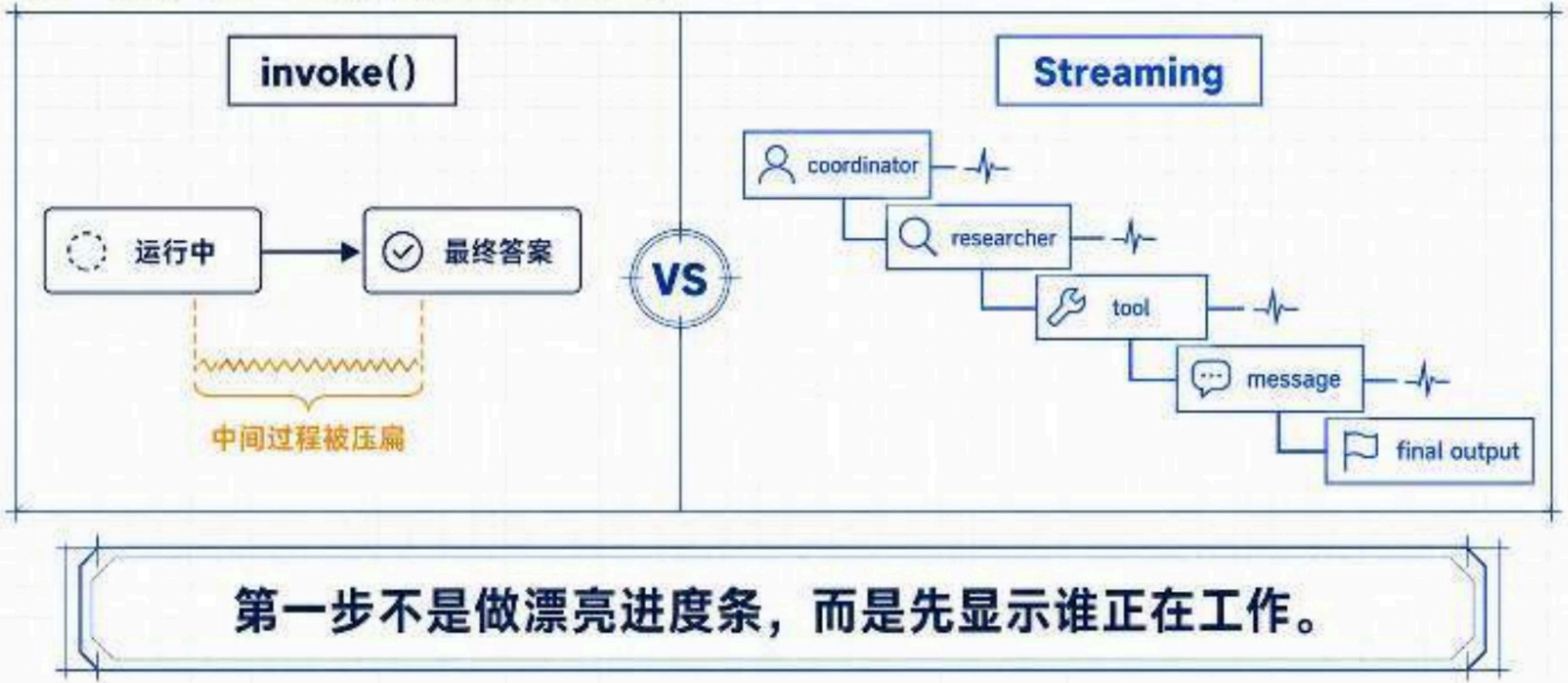
1. 委派是否发生？

2. 工具正在运行，还是已经失败？

3. 中间过程去了哪里？

invoke() 只交付结果，Streaming 交付运行过程

同一请求，体验差异来自中间事件是否可见



第一步不是做漂亮进度条，而是先显示谁正在工作。

运行 Streaming：v3 看对象，v2 看事件包

每个版本只保留一个入口，并锁定三个真正需要消费的点

v3 · 产品视图

```
stream = agent.stream_events(..., version="v3")
```

- **stream.subagents**
  找到委派；读取 path / status
- **subagent.messages**
  进入这个 subagent 自己的消息流
- **message.text**
  拿到可直接展示的文本

v2 · 图执行协议

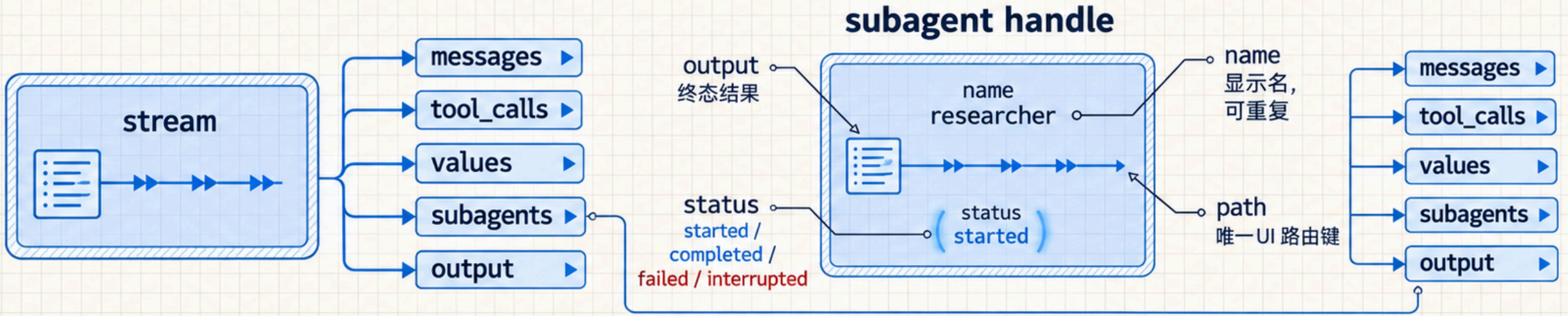
```
agent.stream(..., stream_mode="updates",
    subgraphs=True, version="v2")
```

- **chunk["type"]**
  先筛 updates，忽略其他类型
- **chunk["ns"]**
  空路径 = main；非空路径 = subagent
- **chunk["data"]**
  完成路由后，再解释 payload

≠

两个循环分开消费，再在应用层统一

Projection 按需消费：先订阅，再推进运行



❶ iter(...)、for 或 interleave(...) 建立订阅 → ❷ 事件进入对应消费者缓冲 → ❸ 消费增量并等待终态

⚠ 仅访问属性 ≠ 已订阅。

⚠ 先读取 stream.output 再订阅，可能漏掉 handle。

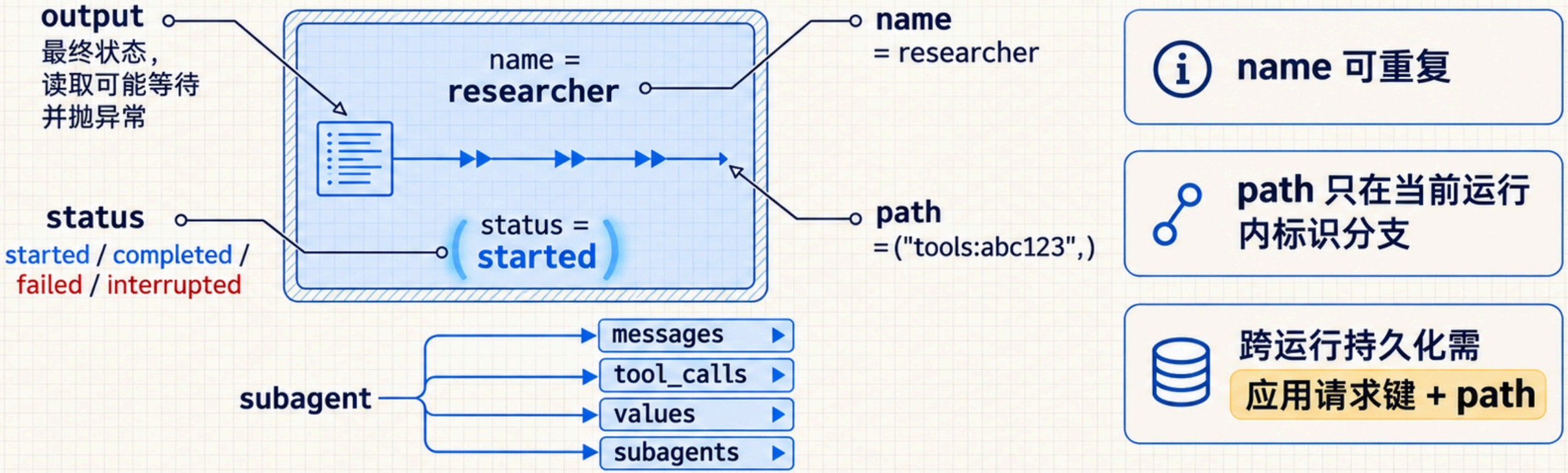
values

- values 是状态快照，不是消息增量；
- 按路径覆盖当前快照。

v0.7 默认无 Todo；
需要时显式启用 TodoListMiddleware。

ⓘ 版本升级后重跑订阅时机实验。

subagent handle：名字用于显示，路径用于路由

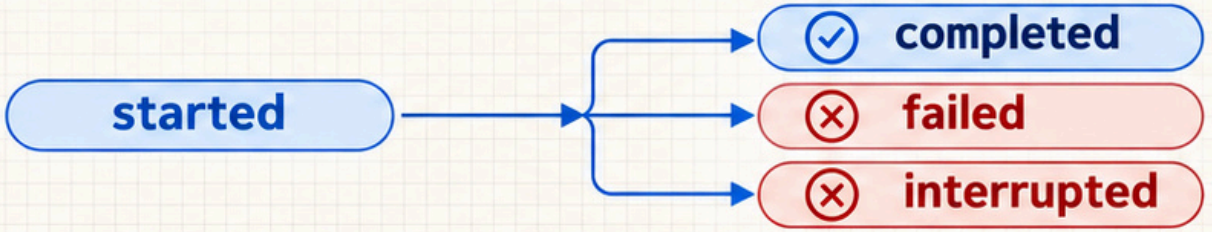


name 可重复

path 只在当前运行内标识分支

跨运行持久化需
应用请求键 + path

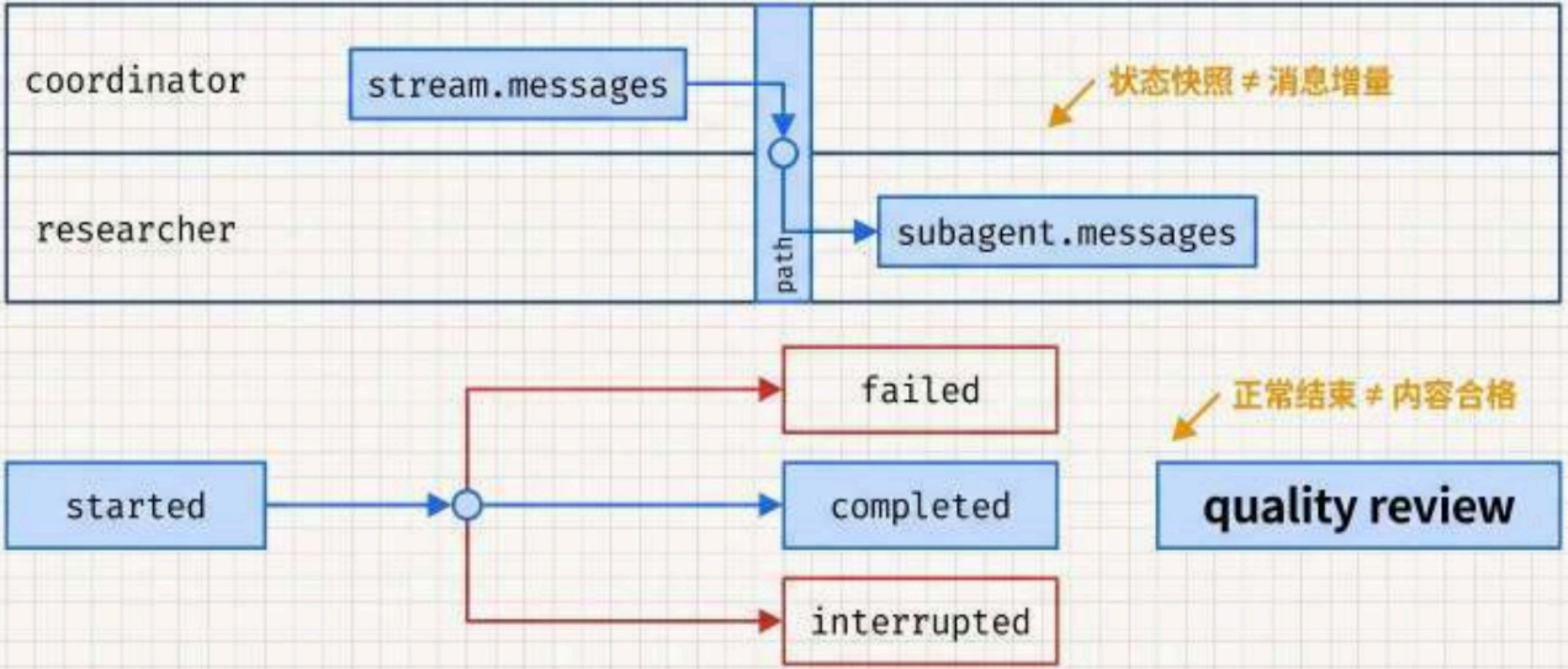
生命周期 (status)



⚠ **completed** 只表示正常结束，不代表内容通过验收。

Projection 不会替你跨层合并，completed 也不是质量结论

coordinator 与 researcher 是两条独立事件范围



message.text 是文本 projection，不是普通字符串

同步完整文本 ⚠ 等待当前消息完成

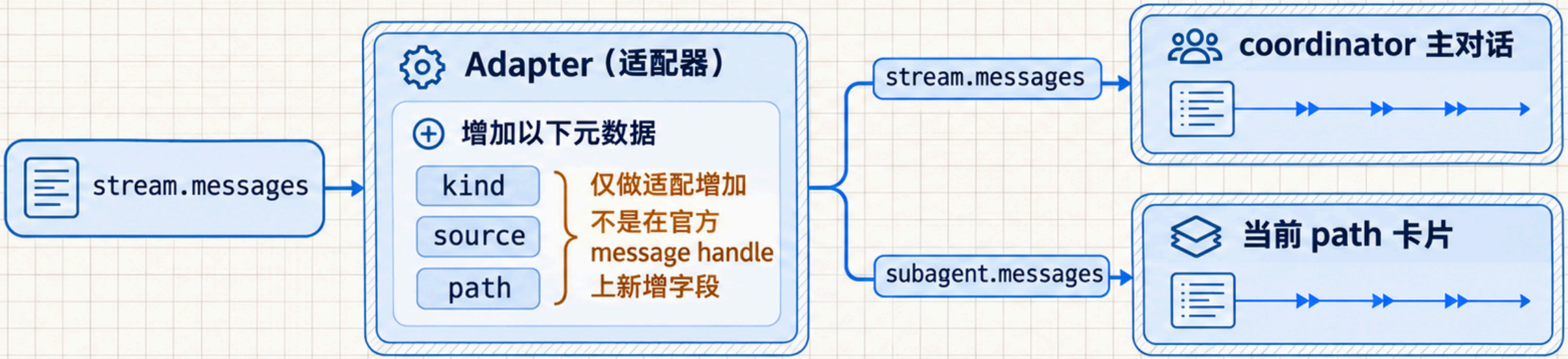
```
str(message.text)
```

异步完整文本 ⚠ 等待当前消息完成

```
await message.text
```

逐段显示

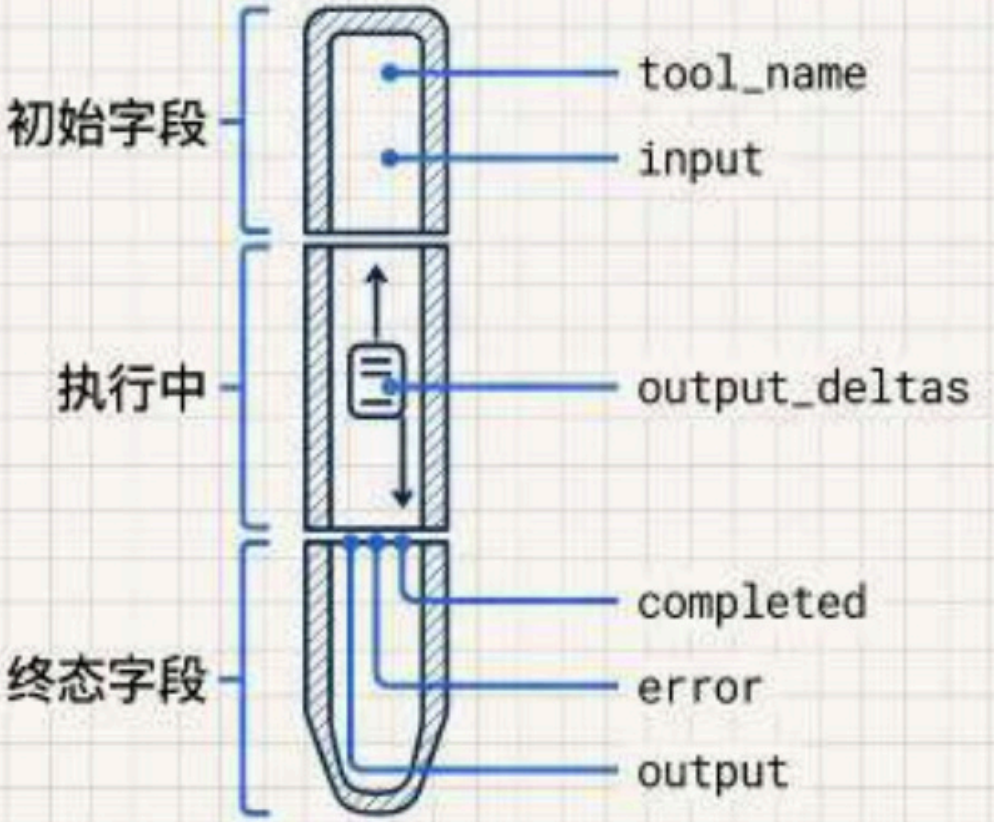
迭代文本增量

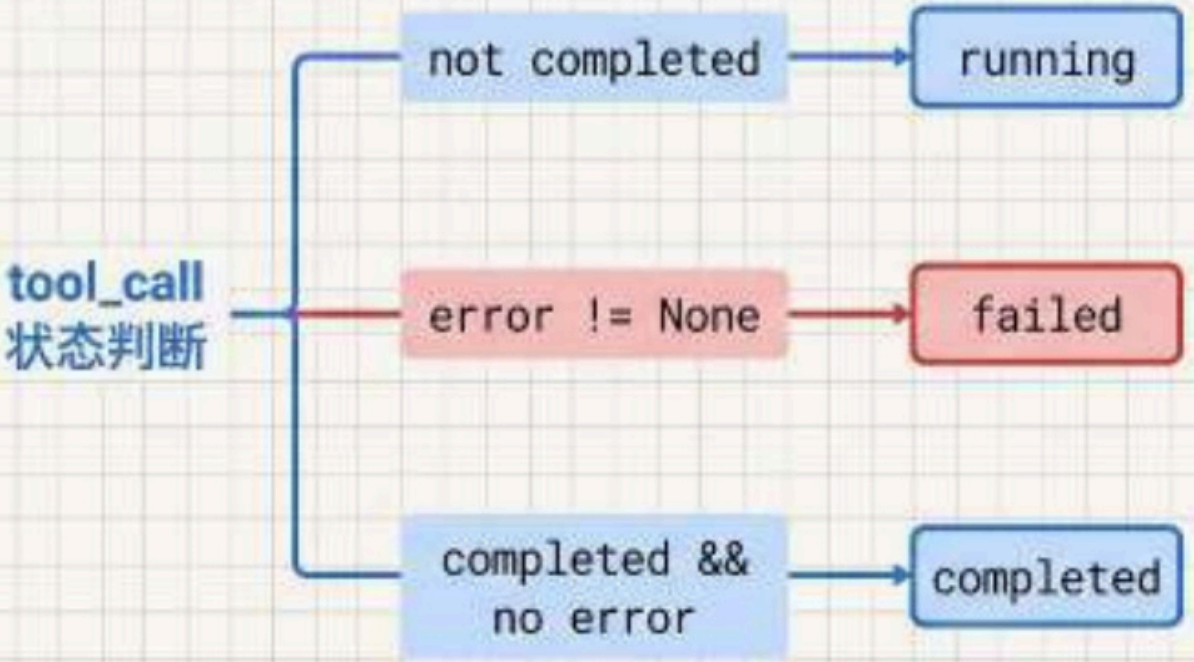


⚠ 完整文本读取和内层循环可能阻塞外层展示；**不要据此声称全树 token 精确排序。**

tool_call 必须同时判断 completed 和 error

增量、终态与失败是三个不同时间点





不要把 delta 与最终 output 重复追加

晚订阅不只是乱序：还可能漏掉整个过程

错误流程

消费 coordinator 消息 → 图谱已完成 → 订阅子 agent（subagents） → 未收到任何 handle

⚠ 先耗尽主流，再订阅子流，可能不是“晚展示”，而是“收不到”

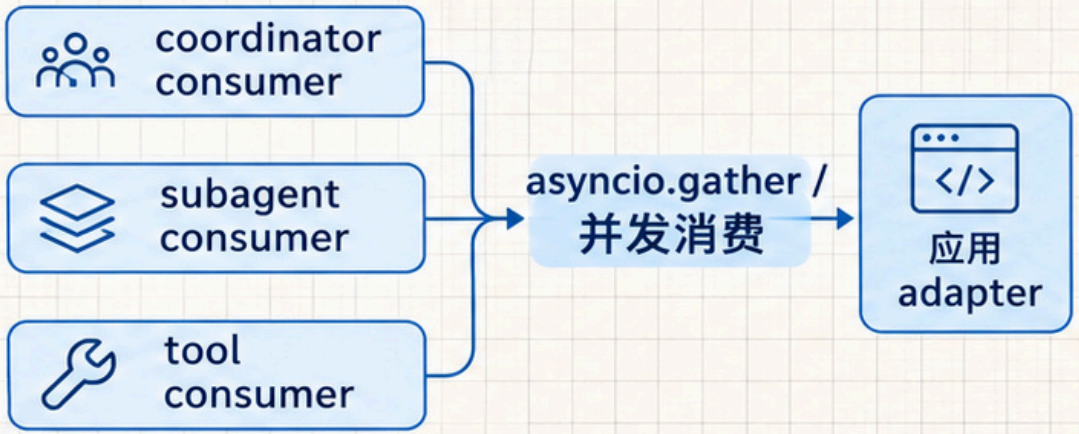
正确流程

先订阅所需的投影 → 收到 researcher handle → 及时消费子投影 → 保留生命周期与工具证据

⚠ 当前事件通道按订阅消费；内存 iterator 不是可回放日志。

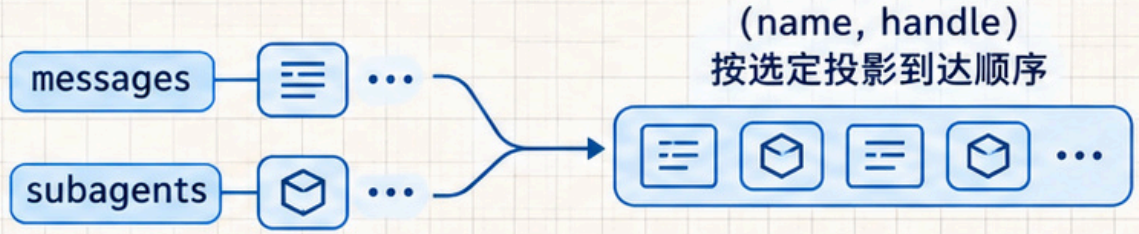
并发与 interleave：解决不同层面的等待

异步并发：多源同时消费



同步交织：按投影顺序合并

```
stream.interleave("messages", "subagents")
```



⚠ str(item.text) 和内层循环仍会阻塞外层。

三个规则

1. 只合并显式选择的 projection；
2. 不会自动合并全部子层 token；
3. 全协议顺序看 raw seq。

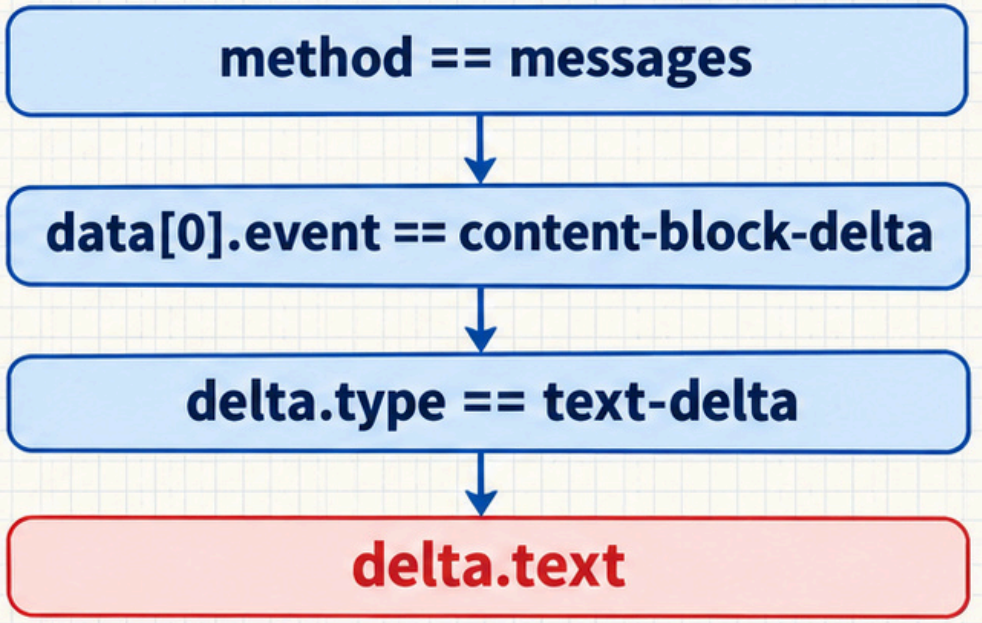
Streaming ≠ 并行执行；编排决定并行性。

raw events：用 seq 排序，用 namespace 路由

事件结构（示例）

字段	值
event["seq"]	42
event["method"]	messages
event["params"]["namespace"]	["tools:abc123", "model_request:def456"]
event["params"]["timestamp"]	展示时间
event["params"]["data"]	按 method 解析

过滤链（概念字段检查）



要点

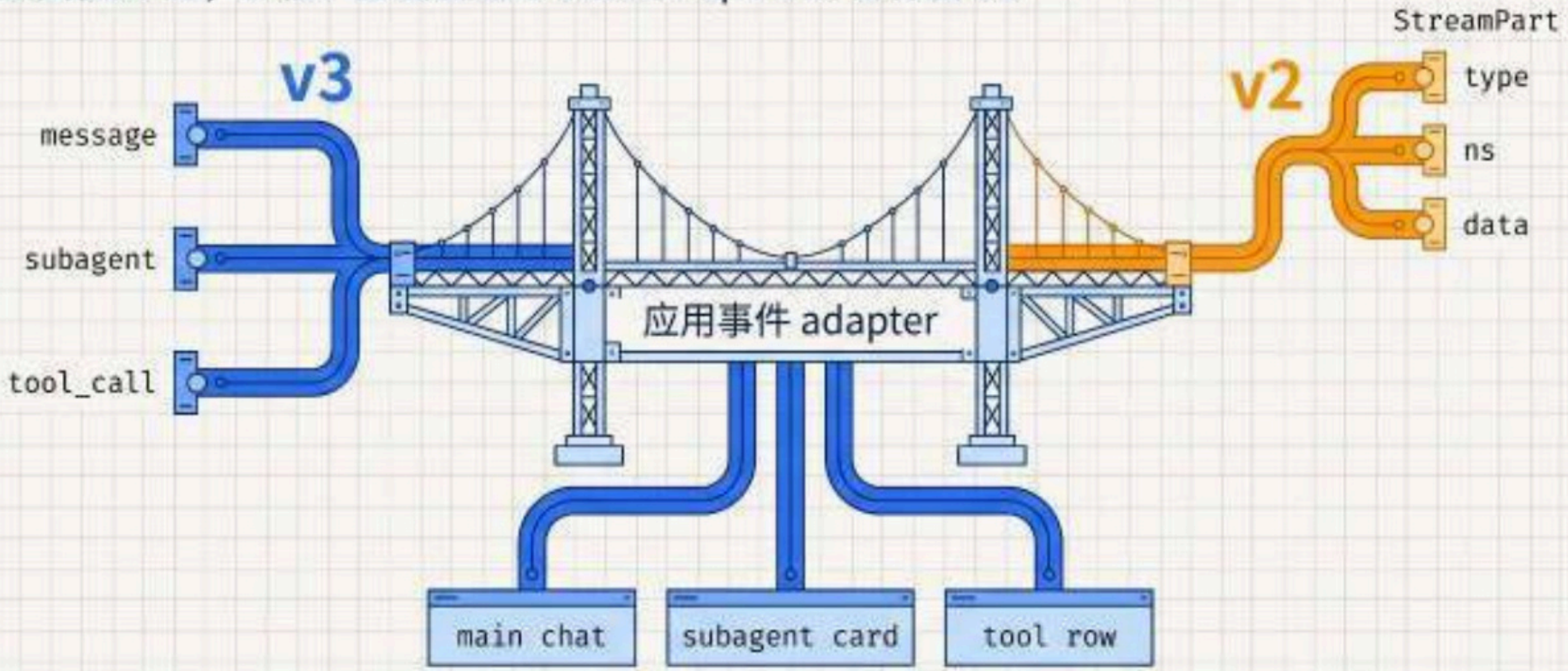
- seq 是同一次运行内严格递增序号；
- timestamp 不保证严格顺序。

⚠ 注意

- 部分模型发出完整 AIMessage；过滤器无文本增量不代表没有输出。
- namespace 是 node:id 路径，不是 researcher 显示名。

v3 是产品视图，v2 是图执行协议

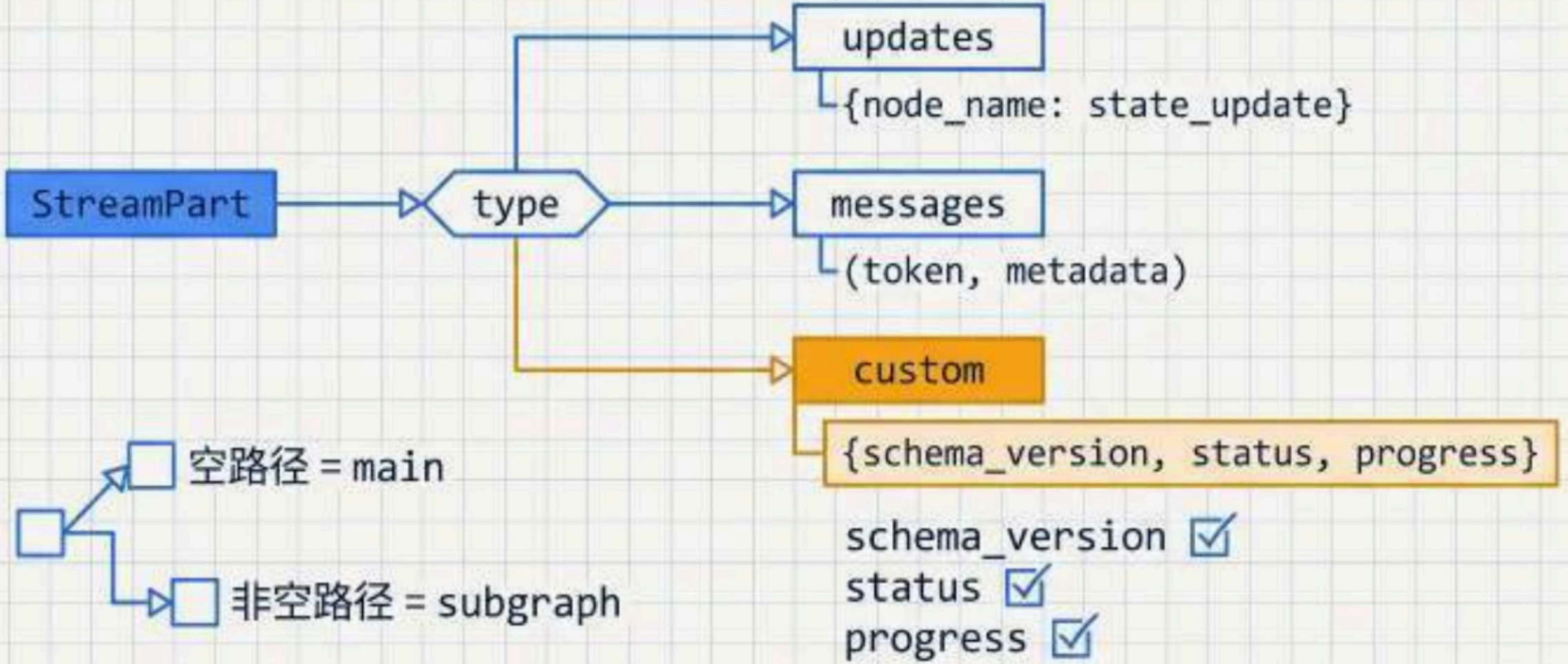
新页面优先 v3；迁移、底层调试与 custom updates 才保留 v2



不要混在同一个循环

v2 先看 type，再按 ns 路由，最后解释 data

data 没有固定形状；custom 事件更需要自定义 schema



可复现实验：先离线验证，再接真实模型

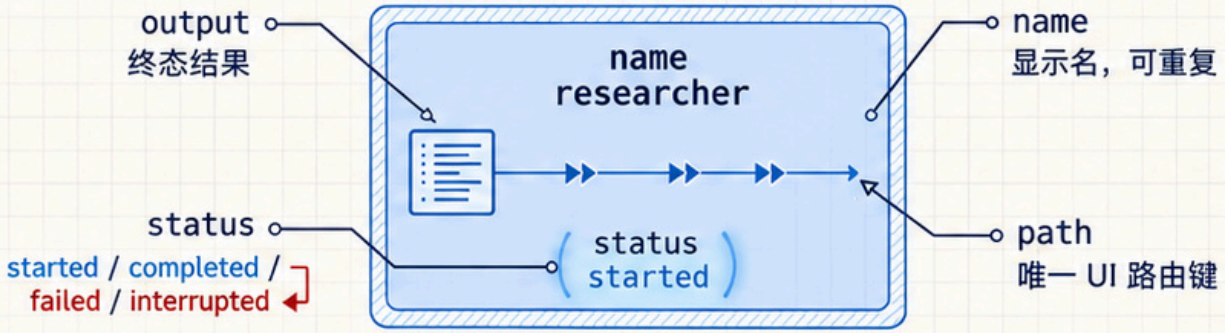
OFFLINE：离线验证（已知 Notebook）

```
> uv run --project notebooks --locked \
  python -m course_notebooks.run ch14-streaming
```

- 无需模型密钥（No Model Key）
- ScriptedChatModel + 真实图、委派与本地工具

⚠ 不支持 token 流式输出

⚠ 不用于证明真实模型的质量



版本说明： 当前 Notebook 锁：deepagents 0.7.22 / langgraph 1.2.13；正文订阅说明引用：0.7.15 / 1.2.11。

OPTIONAL TEMPLATE：项目模板（可选）

```
> agentseek create deepagents/streaming \
  --checkout main --no-input
```

初始化后执行建议

- README 配置（环境与运行说明）
- task --list（查看可运行任务）
- doctor（环境与依赖自检）
- >_ dev（本地开发运行）

验收：委派、独立状态、工具结果、raw seq、主 Agent 汇总；不比固定文案。

从看见过程，到可靠交付

断开与超时：
明确取消，或转后台继续

背压：
限制队列，避免慢客户端
无限积压

重放：
标准化事件写入 Trace
或持久化存储

失败：
工具错误、子 Agent
failed/interrupted 原样保留



Streaming 不自动提供持久化、取消、超时或背压；输入与工具结果先脱敏。

path 是分支根，namespace 是事件位置

协调树（以一个运行内的调用为例）

根 coordinator
v2 ns=() / v3 namespace=[]

↓

researcher
path=("tools:abc123",)

↓

inner model
v3 namespace=[
"tools:abc123", "model_request:def456"
]

关键关系：前缀包含，而非必须相等

⚠ **path 是 namespace 的前缀，但不一定相等。**

✓ 例：path=("tools:abc123",)
✓ 例：namespace=["tools:abc123","model_request:def456"]
⇒ namespace 以 path 为前缀，属于该子代理。

判断归属的代码

```
def belongs_to_subagent(namespace, path):
    return tuple(namespace[:len(path)]) == path
```

同名 researcher 可以多次运行；
嵌套委派**保留完整路径**，不只取第一个 tools ID。

范围： 当前运行内路由；
跨运行增加应用请求键。

三层字段别混用：框架、协议、应用 adapter

框架：typed handles

框架结构化字段 (typed)

- name
- path
- status
- messages
- tool_calls
- values
- output

⚠ 注意：这是框架的类型化字段，不要直接当作协议字段或业务字段。

协议：raw protocol

协议传输字段 (raw)

- seq
- method
- params.namespace
- params.timestamp
- params.data

⚠ 注意：这是协议的原始字段，通常用于传输、排序与解析。

应用：app adapter

应用适配字段 (adapter)

- kind
- source
- phase
- `path` 转成 JSON list
- `sequence` 来自 raw seq

⊗ 失败：把 adapter 字段当作框架或协议字段会导致语义混乱、解析错误。

框架（结构化处理） → **协议（原始传输）** → **应用（业务适配）**

关键说明

- `kind` / `source` / `phase` 由应用生成，不是框架自动附带。
- `thread_id` 用于会话连续性，不是流事件 ID。
- custom 的 `status` / `progress` / `schema_version` 由业务定义并验证。

模板字段按版本核对；不要凭空宣称官方有 `run_id`。